

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук

Кафедра программирования и информационных технологий

Сервис для подбора недвижимости Sweet Neighbors Club

Технологии программирования

Курсовая работа

09.03.02 Информационные системы и технологии

Обработка информации и машинное обучение

Зав. кафедрой _____ д.ф-м.н., доцент С.Д. Махортов

Руководитель _____ к.т.н., ст. преп С.Н. Шилов

Руководитель практики _____ асс. М.А. Савин

Обучающийся _____ ст. 3 курса И.Н. Гелемеев

Обучающийся _____ ст. 3 курса Р.Д. Холопов

Обучающийся _____ ст. 3 курса С.А. Хромых

Воронеж 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Теоретическая часть	5
1.1 Анализ предметной области	5
1.1.1 Особенности рынка недвижимости	5
1.1.2 Проблемы традиционного поиска недвижимости	5
1.2 Анализ существующих решений	6
1.2.1 Авито Недвижимость.....	6
1.2.2 Циан	7
1.2.3 Домклик.....	7
1.3 Постановка задачи	8
1.3.1 Описание системы	8
1.3.2 Назначение и цели системы	9
1.3.3 Финансовая модель.....	9
2 Практическая часть	9
2.1 Требования к системе	9
2.2 Сценарии использования системы	11
2.2.1 Требования к ПО.....	13
2.2.2 Архитектура системы	14
2.2.3 Метрики успеха.....	16
2.3 Выбор методологии разработки	16
2.3.1 Применение практики Agile	17
2.3.2 Ожидаемые преимущества для данного проекта	18
2.4 Проектирование базы данных.....	19
2.4.1 Логика работы предметной области через схему БД.....	20
2.5 Разработка REST API	21
2.5.1 Аутентификация, авторизация и общая структура маршрутов.....	21
2.6 Разработка скраперов	22
2.7 Интерфейс	24
2.8 Разработка PWA приложения	26
2.9 Тестирование	28
ЗАКЛЮЧЕНИЕ	31
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33

ВВЕДЕНИЕ

Современный рынок недвижимости в России характеризуется высокой динамикой и значительным объёмом предложений. Ежедневно на таких крупных агрегаторах, как «Авито», «Циан» и «Домклик», появляются тысячи новых объявлений о продаже и аренде жилья. Потенциальные покупатели и арендаторы сталкиваются с необходимостью регулярно просматривать несколько веб-ресурсов, чтобы не упустить подходящий вариант. При этом ликвидные объекты, особенно с привлекательной ценой, могут быть сняты с публикации в течение нескольких часов после размещения. Ручной мониторинг становится не только трудоёмким, но и неэффективным, поскольку высокая скорость обновления информации и разрозненность источников приводят к пропуску релевантных предложений.

Существующие агрегаторы предлагают встроенные инструменты уведомлений, однако они ограничены рамками одного сервиса и не позволяют пользователю получать единую картину со всех значимых площадок. Кроме того, функциональность фильтрации часто не соответствует индивидуальным потребностям пользователя, а гибкость настройки параметров поиска оставляет желать лучшего. Таким образом, задача создания универсального сервиса, который объединяет данные из нескольких источников, предоставляет расширенные возможности фильтрации и оперативно уведомляет пользователя о новых объявлениях, является актуальной.

Целью данной работы является проектирование и описание архитектуры информационной системы мониторинга объявлений недвижимости, которая автоматизирует процесс сбора, обработки и доставки пользователям релевантных предложений с популярных веб-ресурсов.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области и выявить особенности рынка недвижимости, влияющие на процесс поиска.

2. Выполнить обзор существующих решений для мониторинга объявлений, определить их достоинства и недостатки.

3. Сформулировать функциональные и нефункциональные требования к разрабатываемой системе.

4. Обосновать выбор технологического стека и архитектурных решений.

5. Описать состав работ и этапы реализации проекта.

6. Реализовать приложение для поиска недвижимости.

Объектом исследования выступает процесс поиска объектов недвижимости на специализированных веб-ресурсах. Предметом исследования являются методы и средства автоматизации сбора, обработки и анализа данных объявлений.

1 Теоретическая часть

1.1 Анализ предметной области

1.1.1 Особенности рынка недвижимости

Рынок недвижимости является одним из наиболее консервативных, но при этом высококонкурентных сегментов цифровой экономики. Его ключевые особенности включают:

1. Высокую стоимость объектов, что заставляет пользователей тщательно анализировать предложения;
2. Сезонность спроса, проявляющуюся в увеличении количества сделок в весенне-осенний период;
3. Неоднородность данных - объявления могут содержать различный набор полей (площадь, этаж, материал стен, наличие ремонта и т.д.), что затрудняет автоматизированное сравнение;
4. Быструю оборачиваемость ликвидных объектов, когда время между публикацией и заключением сделки может составлять менее суток.

1.1.2 Проблемы традиционного поиска недвижимости

Пользователи, не использующие специализированные инструменты мониторинга, сталкиваются со следующими трудностями:

1. Необходимость ручного просмотра нескольких сайтов — ни один из существующих агрегаторов не охватывает все источники данных.
2. Информационная перегрузка - большое количество объявлений, не соответствующих критериям, требует значительных временных затрат на фильтрацию.
3. Высокий риск пропуска выгодного предложения - отсутствие мгновенных уведомлений о новых публикациях приводит к тому, что пользователь узнаёт об объявлении уже после его снятия.

4. Ограниченные возможности фильтрации - встроенные фильтры часто не позволяют задать сложные условия (например, поиск квартиры в конкретном районе).

Таким образом, автоматизация процесса сбора и фильтрации объявлений с возможностью оперативного уведомления является объективной необходимостью для повышения эффективности поиска недвижимости.

1.2 Анализ существующих решений

На российском рынке доминируют три крупнейших агрегатора объявлений о недвижимости: «Авито Недвижимость», «Циан» и «Домклик». Каждый из них предоставляет пользователям возможность поиска и настройки уведомлений, однако подходы и функциональность различаются.

1.2.1 Авито Недвижимость

Описание: «Авито» является крупнейшей онлайн-площадкой объявлений в России. Раздел «Недвижимость» включает предложения от собственников и агентств.

Фильтры: тип операции (продажа, аренда, посуточно); тип недвижимости (квартира, комната, дом, участок и др.); количество комнат; диапазон цены; площадь (общая, кухни); этаж и этажность дома; тип дома (панельный, кирпичный, монолитный); наличие ремонта; год постройки дома; вид парковки; наличие ремонта; просмотр отзывов жильцов и проверка данных из Росреестра.

Уведомления: возможность подписаться на рассылку; уведомления приходят на электронную почту и в мобильное приложение (push); частота уведомлений: ежедневно или моментально (в приложении); нельзя настроить уведомления только по новым объявлениям от собственников без агентств.

Ограничения: нет возможности мониторинга других площадок; в бесплатной версии уведомления не всегда приходят мгновенно; API для внешнего доступа к данным отсутствует. [1]

1.2.2 Циан

Описание: «Циан» — специализированный агрегатор недвижимости, ориентированный в первую очередь на рынок жилой недвижимости. Отличается детализированными карточками объектов и развитой аналитикой.

Фильтры: все стандартные фильтры (цена, комнаты, метро); расстояние до метро; расширенный набор параметров: тип перепланировки, наличие балкона, высота потолков, тип санузла; фильтр по новостройкам с выбором стадии строительства; возможность исключить объявления от агентств; фильтр по ID объекта (для повторного поиска).

Уведомления: «Автопоиск» - рассылка на email и в приложение; возможность подписаться на поиск с заданными параметрами; настройка периода (моментально, раз в день, раз в неделю); возможность подписаться на новостройки по ЖК.

Ограничения: отсутствие интеграции с другими агрегаторами; многие расширенные фильтры доступны только в мобильном приложении; [2]

1.2.3 Домклик

Описание: «Домклик» - сервис от Сбера, ориентированный на ипотечные сделки и вторичный рынок. Данные поступают в том числе от партнёров банка.

Фильтры: стандартные параметры (цена, площадь, комнаты); фильтр по ипотечной ставке и возможности одобрения; проверка юридической чистоты (отслеживание залогов, обременений); удалённость от офисов банка.

Уведомления: подписка на новые объявления по параметрам поиска; уведомления в мобильном приложении и по email; есть рекомендации на основе просмотров.

Ограничения: база данных ориентирована на клиентов Сбера, охват меньше, чем у «Авито» или «Циан»; отсутствие расширенных фильтров по состоянию ремонта, типу перепланировки; уведомления могут дублироваться, нет гибкой настройки каналов.

Все рассмотренные решения обладают одним ключевым недостатком — они ограничены данными собственной площадки. Пользователь вынужден либо выбирать один агрегатор, что снижает вероятность найти наилучшее предложение, либо настраивать уведомления на нескольких сервисах, что приводит к информационному шуму и усложняет управление.

Разрабатываемая система призвана устранить этот недостаток за счёт: объединения данных из трёх основных источников («Авито», «Циан», «Домклик»), что даёт пользователю наиболее полную картину рынка; единых параметров фильтрации, позволяющих задать условия один раз и применять их ко всем источникам; централизованного управления уведомлениями через удобный веб-интерфейс и PWA. [3]

1.3 Постановка задачи

1.3.1 Описание системы

Разрабатываемая система представляет собой сервис мониторинга объявлений недвижимости, который в автоматическом режиме собирает данные с внешних веб-ресурсов («Авито», «Циан», «Домклик»), обрабатывает их, сохраняет в базе данных и предоставляет пользователям возможность настройки персонализированных фильтров. При появлении новых объявлений, соответствующих заданным критериям, система отправляет уведомления на мобильное устройство и/или электронную почту пользователя.

Система реализуется в виде веб-приложения с поддержкой Progressive Web App (PWA), что позволяет использовать её как на настольных компьютерах, так и на мобильных устройствах без необходимости установки нативного приложения.

1.3.2 Назначение и цели системы

Назначение системы — автоматизация процесса сбора, обработки и анализа объявлений о недвижимости с внешних веб-ресурсов для упрощения поиска подходящих объектов.

Цель системы — повышение эффективности пользователей при поиске недвижимости за счёт:

1. автоматического мониторинга нескольких источников данных;
2. гибкой настройки параметров поиска;
3. своевременной доставки уведомлений о новых релевантных объявлениях.

1.3.3 Финансовая модель

Сервис реализуется по модели freemium:

Пользователь получает бесплатный доступ на неделю с полной функциональностью. После окончания пробного периода доступ осуществляется по подписке. Стоимость подписки составляет 200 рублей в месяц.

Такая модель позволяет привлечь максимальное количество пользователей на этапе запуска и обеспечить стабильный доход для поддержания и развития сервиса.

2 Практическая часть

2.1 Требования к системе

Функциональные требования:

1. FR-1: Система должна выполнять автоматический сбор данных с сайтов недвижимости.
2. FR-2: Система должна извлекать данные из JSON-ответов, используемых фронтендом сайтов.
3. FR-3: Система должна сохранять объявления в базе данных.
4. FR-4: Система должна предоставлять пользователю возможность задавать следующие параметры поиска:
 - Тип недвижимости (дом, квартира, комната и т.д.);
 - Количество квадратных метров;
 - Количество комнат;
 - Этаж;
 - Создатель объявления (агентство или собственник);
 - Цена;
 - Год постройки дома;
 - Наличие ремонта;
 - Условия проживания.
5. FR-5: Система должна фильтровать объявления в соответствии с параметрами пользователя.
6. FR-6: Система должна отправлять уведомления на мобильное устройство.
7. FR-7: Система должна отправлять уведомления по электронной почте.
8. FR-8: Система должна предоставлять веб-интерфейс для управления настройками.
9. FR-9: Система должна поддерживать регистрацию, аутентификацию и авторизацию пользователей.

10. FR-10: Система должна предоставлять пользователю возможность посмотреть объявления, которые уже находятся на рынке.

11. FR-11: Система должна предоставлять пользователю ссылки на оригинальные объявления на агрегаторах недвижимости, а не полноценные карточки товаров.

12. FR-12: Система должна предоставлять пользователю фотографии с оригинальной карточки объекта.

13. FR-13: Система должна предоставлять пользователю базовые характеристики объекта в веб-интерфейсе.

14. FR-14: Система должна кэшировать недавно полученные объявления для проверки дубликатов.

Нефункциональные требования:

1. NFR-1: Обновление данных должно выполняться не реже одного раза в 10 минут.

2. NFR-2: Система должна обеспечивать доступность не менее 90% времени.

3. NFR-3: Время обработки одного источника не должно превышать 5 секунд.

4. NFR-4: Система должна быть масштабируемой для подключения новых источников.

5. NFR-5: При проблемах с обработкой одного из источников система должна оставаться работоспособной.

2.2 Сценарии использования системы

Приведём основные сценарии использования системы для подбора недвижимости. Акторы системы: Пользователь, Система, Сервис Push-уведомлений, Email-сервис, Внешний сайт недвижимости.

UC-1. Регистрация и аутентификация в системе

Акторы: Пользователь, Система
Цель: получить доступ к личному кабинету системы.
Предусловия: у пользователя есть доступ к веб-интерфейсу.

Основной поток:

1. Пользователь нажимает кнопку входа/регистрации в веб-интерфейсе.
2. Пользователь вводит учётные данные (логин/пароль).
3. Пользователь нажимает кнопку «Войти» или «Зарегистрироваться».
4. Система проверяет данные и авторизует пользователя.

Негативный сценарий: при вводе неверного пароля или логина Система выводит сообщение об ошибке. Пользователь может попробовать снова или восстановить пароль.

Постусловия: Пользователь авторизован и может использовать веб-интерфейс для настроек.

УС-2. Настройка параметров поиска недвижимости

Акторы: Пользователь, Система
Цель: задать критерии, по которым система будет искать подходящие объекты.
Предусловия: Пользователь авторизован в системе.

Основной поток:

1. Пользователь открывает веб-интерфейс управления настройками.
2. Пользователь задаёт желаемые параметры поиска: цена, площадь, город, количество комнат, этаж, тип недвижимости, наличие ремонта, год постройки и др.
3. Пользователь нажимает кнопку «Сохранить».
4. Система сохраняет параметры в профиле пользователя.

Негативный сценарий: при некорректном вводе (например, минимальная цена больше максимальной) Система просит исправить ввод.

Постусловия: Параметры поиска сохранены и готовы к использованию модулем фильтрации.

УС-3. Фильтрация объявлений и отправка уведомлений

Акторы: Система, Сервис Push-уведомлений, Email-сервис

Цель: оповестить пользователя о появлении объектов, соответствующих его интересам.

Предусловия: Завершился процесс сбора данных (парсинг), в базе появились новые объявления.

Основной поток:

1. Система берёт новые объявления и фильтрует их в соответствии с сохранёнными параметрами пользователя.
2. Система формирует список подходящих объявлений для конкретного пользователя.
3. Система отправляет уведомление о новых объектах на мобильное устройство пользователя (push).
4. Система отправляет дублирующее уведомление на электронную почту.

Альтернативный поток: если пользователь в настройках отключил один из каналов, система отправляет только через включённые каналы.

Постусловия: Пользователь проинформирован о появлении на рынке подходящих объектов.

2.2.1 Требования к ПО

Технологический стек:

1. Backend: Python (FastAPI) - выбран за высокую производительность, асинхронную поддержку и автоматическую генерацию OpenAPI документации, что ускоряет разработку и интеграцию с фронтендом.
2. База данных: PostgreSQL - используется как надёжная реляционная СУБД с поддержкой сложных запросов и транзакций, что критически важно для корректной работы фильтрации.

3. Очереди задач: Celery - применяется для выполнения фоновых задач (скрапинг, отправка уведомлений) с возможностью распределённой обработки и повторных попыток при сбоях.

4. Кэш и брокер сообщений: Redis - используется в качестве брокера сообщений для Celery, обеспечивая быструю передачу задач между компонентами системы. Кроме того, Redis применяется для кэширования часто запрашиваемых данных и хранения временных состояний (например, уже обработанных объявлений), что позволяет снизить нагрузку на основную базу данных и повысить общую производительность системы.

5. Frontend: React - обеспечивает создание динамического пользовательского интерфейса с повторно используемыми компонентами и высокой скоростью рендеринга.

6. Мобильная версия: Progressive Web App (PWA) - позволяет обеспечить доступ с мобильных устройств с возможностью получения push-уведомлений без разработки нативного приложения.

2.2.2 Архитектура системы

Архитектура системы представлена на рисунке 1.

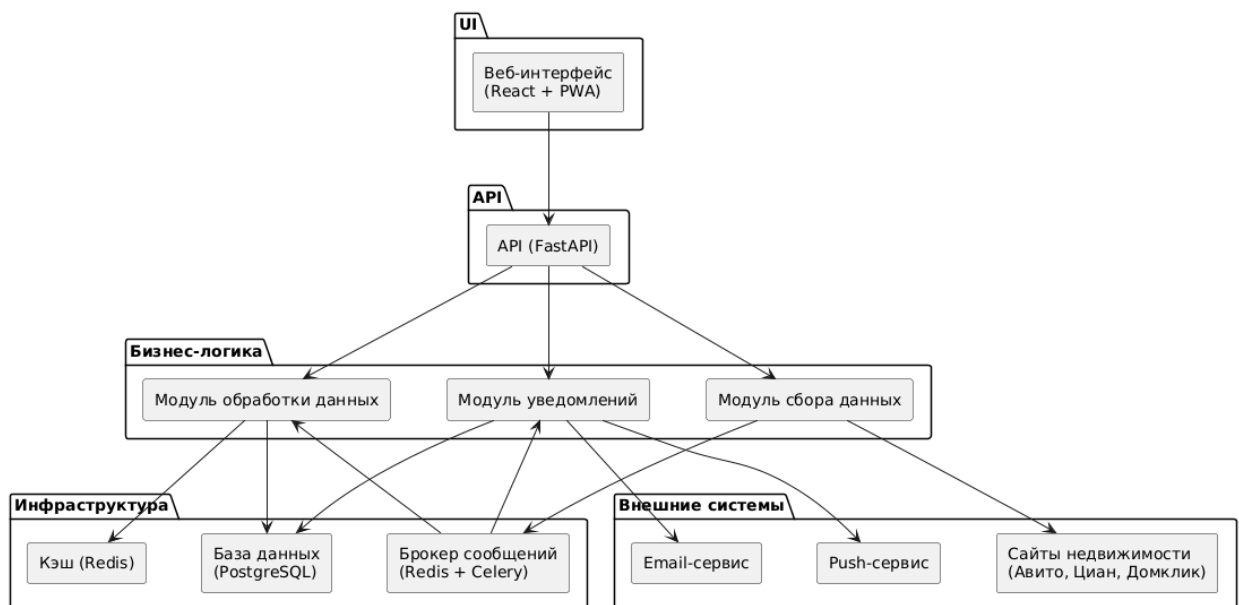


Рисунок 1 - Архитектура системы

Данная система реализует следующие компоненты:

1. Модуль сбора данных - выполняет периодические запросы к источникам, извлекает данные из JSON-ответов и передаёт их в очередь на обработку.
2. Модуль обработки данных - нормализует полученные данные, приводит к единому формату, проверяет дубликаты и сохраняет в БД.
3. База данных - хранит пользователей, параметры фильтрации, объявления и историю уведомлений.
4. API - предоставляет интерфейс для взаимодействия фронтенда с базой данных, реализует бизнес-логику фильтрации и управления пользователями.
5. Брокер сообщений - используется для организации очередей задач и передачи данных между модулями системы. Обеспечивает асинхронное взаимодействие между модулем сбора данных, обработки и модулем уведомлений.
6. Кэш - применяется для хранения временных данных (например, уже обработанных объявлений), что позволяет ускорить обработку, снизить нагрузку на базу данных и повысить производительность системы.
7. Модуль уведомлений - отслеживает появление новых объявлений, сопоставляет их с подписками пользователей и отправляет уведомления через выбранные каналы (email, push).
8. Веб-интерфейс - обеспечивает взаимодействие пользователя с системой: регистрацию, настройку фильтров, просмотр объявлений.

Общий поток данных:

Сбор (модуль сбора по расписанию опрашивает источники), обработка (данные нормализуются и проверяются), сохранение (новые объявления записываются в БД), фильтрация (модуль уведомлений проверяет, какие подписки соответствуют новым объявлениям), уведомление (отправляются сообщения пользователям).

2.2.3 Метрики успеха

Для оценки эффективности системы будут использоваться следующие метрики:

1. Время доставки уведомления - интервал между появлением объявления на источнике и отправкой уведомления пользователю. За целевое значение принято не более 5 минут.
2. Количество релевантных уведомлений - процент уведомлений, которые привели к переходу на просмотр объявления. За целевое значение принято 25-30%.
3. Количество активных пользователей - число пользователей с активной подпиской или находящихся на пробном периоде. За целевое значение принято не менее 50 активных пользователей.
4. Успешность обработки источников - процент успешно обработанных запросов к источникам данных. За целевое значение принято не менее 85% для «Авито» и 90% для «Циан» и «Домклик».

2.3 Выбор методологии разработки

Для реализации сервиса мониторинга объявлений недвижимости была выбрана гибкая методология Agile в упрощенном варианте, адаптированном под команду из двух разработчиков и жесткие сроки (2 месяца). Обоснование выбора Agile

1. Жёсткие временные рамки. Проект должен быть реализован за 2 месяца (8 недель) силами двух разработчиков. Agile позволяет разбить работу на короткие итерации (спринты) длительностью 2 недели. Это даёт возможность чётко планировать объём задач на каждую итерацию и своевременно корректировать планы при обнаружении отклонений.
2. Неопределённость требований к интеграции с внешними источниками. Поскольку официальные API у «Авито», «Циан» и «Домклик»

отсутствуют, структура данных может меняться в любой момент. Agile предполагает быструю реакцию на такие изменения – задача по адаптации парсера включается в ближайший спринт с высоким приоритетом.

3. Минимальный размер команды. Для двух разработчиков тяжёлые методологии (водопадная, RUP) избыточны. Упрощённый Agile требует лишь лёгкого планирования и ежедневной синхронизации, что позволяет сосредоточиться на программировании.

4. Приоритет работающего продукта над документацией. Цель - действующий сервис. В конце каждого спринта команда демонстрирует руководителю работающий инкремент (например, модуль сбора данных, затем API, затем веб-интерфейс). Это даёт раннюю обратную связь.

5. Адаптация к изменяющимся условиям. При обнаружении антибот-защиты (капча, бан IP) требуется оперативно внедрять ротацию прокси или менять стратегию запросов. Agile позволяет пересматривать приоритеты задач на ежедневной встрече без срыва графика. [4]

2.3.1 Применение практики Agile

Из всего набора практик Agile в проекте используются только две, наиболее ценные для малой команды, представленные в таблице 1.

Таблица 1 - Практики

Практика	Как реализована
Ежедневные встречи	Каждый день (кроме выходных) разработчики созваниваются на 10–15 минут и отвечают на три вопроса: что сделано вчера, что планируется сегодня, какие есть препятствия.
Спринты длительностью 2 недели	Весь срок разработки (8 недель) разбит на 4 спринта. В начале каждого спринта

	команда отбирает задачи из общего бэклога (список требований) и фиксирует цель спринта.
--	---

План спринтов представлен в таблице 2.

Таблица 2 - План спринтов

Спринт	Длительность	Основные задачи
Спринт 0 (подготовка)	3 дня	Настройка среды, выбор библиотек (FastAPI, SQLAlchemy, Celery, Redis).
Спринт 1	2 недели	Проектирование базы данных, сбор базовых каркасов FastAPI и React приложений объектов.
Спринт 2	2 недели	Реализация регистрации и аутентификации с авторизацией. Реализация главной страницы со списком объявлений. Реализация скраппинга с «Циан».
Спринт 3	2 недели	Реализация уведомлений, скраппинга с «ДомКлик» и «Авито». Интеграция PWA в приложение.
Спринт 4	2 недели	Тестирование и написание документации.

2.3.2 Ожидаемые преимущества для данного проекта

- Прозрачность прогресса – после каждого спринта видно, что уже работает.
- Быстрая реакция на изменения – если «Авито» изменит JSON, команда переключается на починку парсера в следующем спринте.

— Снижение рисков – интеграционные проблемы выявляются рано, а не в конце проекта.

— Минимум бюрократии - нет лишних встреч и документов, что критично при ограниченных ресурсах (2 разработчика, 2 месяца).

2.4 Проектирование базы данных

В качестве системы управления базами данных выбрана PostgreSQL. Данное решение обусловлено необходимостью обеспечения целостности ссылок, поддержки сложных запросов с множественными условиями (фильтрация объявлений) и ACID-транзакций. Кроме того, PostgreSQL предоставляет тип данных JSONB, который удобен для хранения полуструктурированных расширенных характеристик объявлений. Выбор также обоснован открытой лицензией, кроссплатформенностью и высокой производительностью при работе с большими объёмами данных. Проектирование схемы базы данных выполнено с учётом требований нормализации до третьей нормальной формы, что позволяет избежать избыточности и аномалий обновления.

Схема спроектированной базы данных в виде ER-диаграммы представлена на рисунке 2.

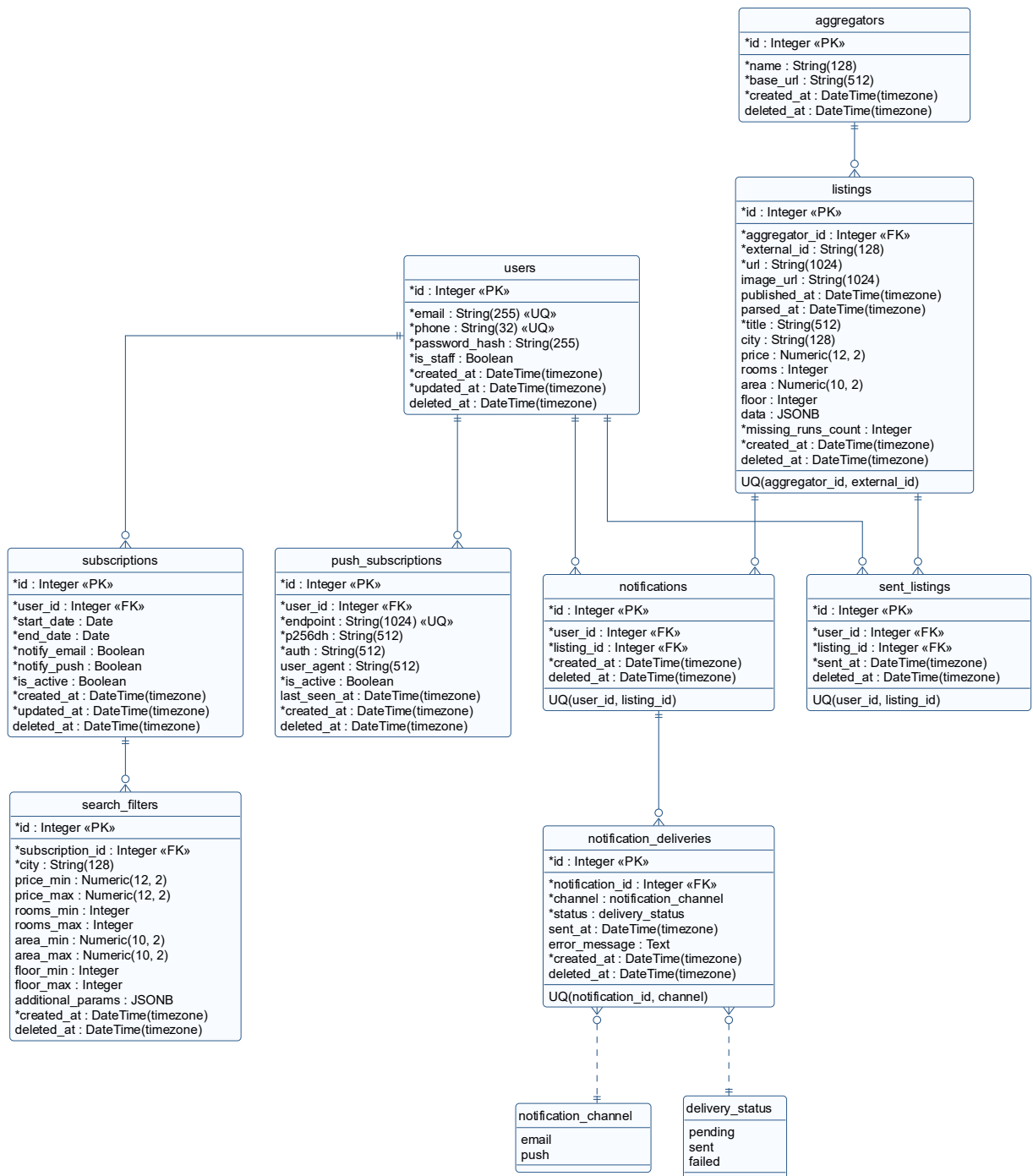


Рисунок 2 - ER-диаграмма

Диаграмма включает все сущности, их атрибуты, первичные и внешние ключи, а также типы связей (один-ко-многим, многие-ко-многим).

2.4.1 Логика работы предметной области через схему БД

1. Пользователь регистрируется → запись в users.

2. После регистрации (или оплаты) создаётся запись в `subscriptions` с указанием дат действия и предпочтений по каналам уведомлений.
3. Пользователь задаёт параметры поиска → они сохраняются в `search_filters`, ссылаясь на конкретную подписку.
4. Система периодически запускает парсинг → объявления попадают в `listings` (с привязкой к `aggregators`).
5. При появлении новых объявлений модуль уведомлений:
 - Сопоставляет каждый новый `listing` со всеми активными `search_filters`.
 - При совпадении создаёт запись в `notifications`.
 - Для каждой записи `notifications` создаёт записи в `notification_deliveries` (по каналам, указанным в `subscriptions`).
6. Для Web Push используется таблица `push_subscriptions`, где хранятся данные для отправки на конкретное устройство.
7. Таблица `sent_listings` служит дополнительным механизмом защиты от повторной отправки (используется как альтернатива или вместе с проверкой уникальности `notifications`).

Такая структура обеспечивает масштабируемость, предотвращает дублирование уведомлений и позволяет гибко расширять набор параметров фильтрации через JSONB-поля.

2.5 Разработка REST API

Взаимодействие клиентской части (React/PWA) с сервером реализовано через REST API, построенное на базе FastAPI. API соответствует принципам RESTful, использует JWT-токены для аутентификации и поддерживает асинхронные фоновые задачи (через Celery).

2.5.1 Аутентификация, авторизация и общая структура маршрутов

Для обеспечения безопасности все защищённые эндпоинты API требуют передачи Bearer-токена в заголовке запроса: Authorization: Bearer <token>.

Токен выдаётся при успешной регистрации или входе пользователя и содержит его идентификатор, срок действия и метку времени. Токен используется для аутентификации (проверки подлинности пользователя) и авторизации (определения его прав).

Особые права (административные) требуются для запуска задач парсинга и управления пайплайном уведомлений. Эндпоинты, помеченные как «требуют прав staff», дополнительно проверяют флаг is_staff в записи пользователя; доступ разрешается только при его значении true.

Все маршруты сгруппированы по функциональному признаку:

Префикс	Назначение
/auth/*	Регистрация, вход, обновление профиля, выход
/listing	Получение списка объявлений (с пагинацией, поиском, сортировкой)
/notifications/*	Управление подписками на уведомления, регистрация push-устройств, запуск пайплайна уведомлений
/scraping/*	Запуск задач парсинга (скрапинга) и контроль их статуса (только для staff)

2.6 Разработка скраперов

В проекте скраперы реализованы как отдельный слой получения данных с сайтов недвижимости. Их задача – обратиться к внешнему источнику, извлечь объявления и привести их к единому внутреннему формату, чтобы

дальше система могла работать с ними независимо от того, откуда они были получены. Общая модель результата описана через `ScrapedListingDTO`: в нее входят внешний идентификатор объявления, ссылка, заголовок, изображение, город, дата публикации, дата парсинга, цена, количество комнат, площадь, этаж и поле для дополнительных данных. Такой формат нужен для унификации, потому что разные площадки отдают данные в разной структуре, но внутри приложения объявления должны храниться и обрабатываться одинаково.

Для скрапперов задан общий контракт `ListingScraper`. Каждый источник должен иметь название агрегатора, базовый адрес сайта и метод `scrape`, который возвращает список нормализованных объявлений. За счет этого основной код не зависит от конкретной реализации Циан, Авито или Домклик: он вызывает один и тот же метод и получает данные в одном формате. Это упрощает сопровождение проекта, потому что изменения в разметке или логике конкретного сайта можно вносить только в соответствующий скраппер, не затрагивая общий механизм сохранения объявлений.

Подключение скрапперов выполнено через автоматический реестр. При запуске система просматривает модули в пакете поставщиков данных, импортирует их, ищет классы с нужными признаками и создает экземпляры подходящих скрапперов. Если указан конкретный источник, например только один агрегатор, реестр отфильтровывает остальные. Если запрошенный источник не найден, выбрасывается ошибка отсутствующего поставщика. Такой подход делает систему расширяемой: для добавления новой площадки нужно создать новый класс с нужными полями и методом сбора, после чего он будет найден автоматически.

Запуск скрапперов выполняется через общий обработчик `run_all_scrapers`. Он создает репозитории для агрегаторов и объявлений, затем последовательно обходит найденные скрапперы. Для каждого источника система получает или создает запись агрегатора в базе, запускает сбор объявлений, преобразует каждое объявление в словарь для сохранения и

передает набор данных в метод массового добавления или обновления. В результате новые объявления создаются, существующие обновляются, а устаревшие могут отмечаться на основе порога пропусков. Это позволяет поддерживать базу в актуальном состоянии без ручной загрузки данных.

Важная часть разработки – обработка ошибок. Каждый скраппер запускается внутри отдельного блока обработки исключений. Если один сайт временно недоступен, изменил структуру страницы или вернул некорректные данные, система фиксирует ошибку, откатывает изменения текущей операции и продолжает работу с другими источниками. В итоговый результат добавляется количество успешно созданных и обновленных объявлений, список выполненных поставщиков и список ошибок. Поэтому сбой одного агрегатора не приводит к остановке всего процесса сбора.

После успешного сохранения хотя бы одного набора объявлений система очищает кэш списка объявлений. Это сделано для того, чтобы пользователь после очередного сбора видел не старую сохраненную выдачу, а обновленные данные из базы. Таким образом, скрапперы связаны не только с загрузкой информации, но и с поддержанием актуальности пользовательской выдачи.

Адреса источников и параметры запросов вынесены в настройки приложения. Для каждого сайта задается адрес поиска, время ожидания ответа, а также при необходимости cookie и пользовательский агент. Это позволяет менять параметры сбора через конфигурацию, не переписывая основной код скрапперов. В проекте уже предусмотрены настройки для Циан, Авито и Домклик, что показывает ориентацию системы на работу с несколькими независимыми площадками недвижимости.

2.7 Интерфейс

Интерфейс приложения ориентирован на быстрый поиск подходящих объявлений и настройку персональных уведомлений. Основная работа

пользователя сосредоточена на странице объявлений и странице параметров уведомлений.

Страница объявлений предназначена для просмотра найденной недвижимости. В ней предусмотрена панель фильтров, где можно задать диапазоны по количеству комнат, цене, площади, этажу и году постройки, а также выбрать тип недвижимости, продавца, наличие ремонта и дополнительные условия. После применения фильтров в основной области отображаются карточки объявлений и навигация по результатам. Данная страница представлена на рисунке 3.

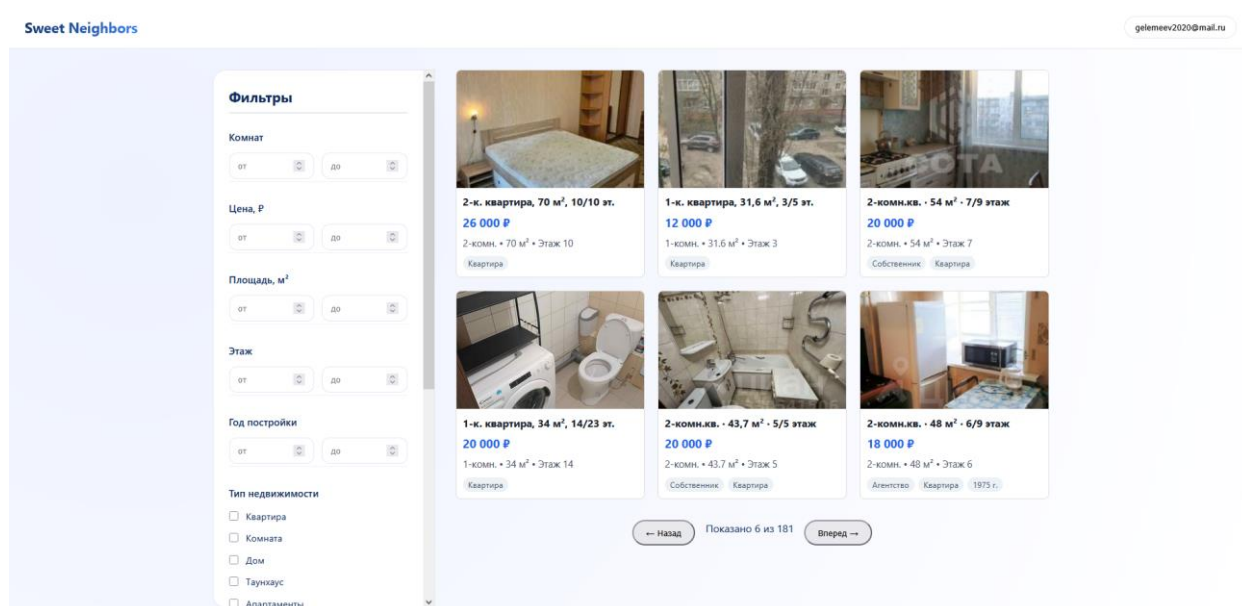


Рисунок 3 - Страница объявлений

Страница настройки уведомлений позволяет пользователю указать, какие объявления должны попадать в рассылку. Здесь выбирается город, канал доставки — электронная почта или push-уведомления, а также критерии поиска: комнаты, цена, площадь, этаж, год постройки, тип недвижимости, продавец и наличие ремонта. После сохранения эти параметры используются системой для автоматического подбора подходящих объявлений и отправки уведомлений. Данная страница представлена на рисунке 4.

Настройки уведомлений

Город
Выберите город
Воронеж

Каналы доставки
☐ Электронная почта
☐ Push-уведомления

Критерии поиска
Комнат
от до

Цена, ₽
от до

Площадь, м²
от до

Этаж
от до

Рисунок 4 - Страница настройки уведомлений

2.8 Разработка PWA приложения

В проекте реализовано прогрессивное веб-приложение (PWA), чтобы пользователь мог работать с сервисом не только как с обычным сайтом, но и как с устанавливаемым приложением. Для этого добавлен манифест приложения: в нем указаны название, короткое название, начальный адрес, область действия, режим отображения без элементов браузера, цветовая схема и иконки разных размеров. Эти параметры позволяют браузеру распознать сайт как приложение и предложить пользователю установить его на устройство.

Отдельный компонент управляет установкой и обновлением приложения. Он перехватывает событие установки, показывает пользователю баннер с кнопкой «Установить», сообщает о доступной новой версии и уведомляет, когда офлайн-режим готов к работе. Также при выходе пользователя очищаются приватные кэши, чтобы данные авторизованного пользователя не сохранялись после завершения сеанса.

Регистрация прогрессивного веб-приложения вынесена в отдельный модуль. В рабочем режиме он подключает фоновый работник (service worker),

который отвечает за обновление приложения и подготовку офлайн-режима. В режиме разработки старые фоновые работники и кэши удаляются, чтобы разработчик не видел устаревшую версию интерфейса.

Дополнительно прогрессивное веб-приложение используется для push-уведомлений. Перед подключением уведомлений система проверяет поддержку браузером уведомлений, фоновых работников и механизма push, затем запрашивает разрешение пользователя и регистрирует подписку. Полученные ключи подписки передаются на сервер, после чего приложение может получать уведомления о новых подходящих объявлениях.

Для обработки входящих push-уведомлений используется отдельный файл фоновых работников. Он принимает данные уведомления, показывает его пользователю и при нажатии открывает нужную страницу приложения. Таким образом, прогрессивное веб-приложение в проекте решает две задачи: делает сервис ближе к обычному мобильному приложению и обеспечивает доставку уведомлений без разработки отдельного нативного клиента.

2.9 Тестирование

Для обеспечения корректности работы разработанного сервиса и предотвращения регрессий при доработках был реализован комплекс автоматических модульных тестов бэкенда. Тестирование охватывает ключевые компоненты: модуль сбора данных (скраперы), репозитории для работы с базой данных, сервисы бизнес-логики, API-эндпоинты, утилиты безопасности и фоновые задачи (Celery).

В качестве фреймворка использовался встроенный в Python unittest в сочетании с unittest.mock для подмены внешних зависимостей (база данных, SMTP-клиент, WebPush, Redis). Для тестирования HTTP-эндпоинтов применялся TestClient из библиотеки FastAPI, позволяющий отправлять запросы к приложению без запуска реального сервера. Все тесты выполняются изолированно, не затрагивая реальные внешние сервисы и базы данных.

Протестированы репозитории базы данных. Проверена логика массовой вставки и обновления объявлений (ListingRepository.upsert_many): корректное увеличение счётчика пропусков missing_runs_count, пометка объявлений как удалённых при достижении порога, обновление существующих записей (test_listing_repository.py). Также протестированы репозитории уведомлений (test_notification_repository_unit.py): создание уведомлений с защитой от гонок (одновременные запросы создают запись только один раз) и материализация каналов доставки без дублирования.

Протестированы сервисы доставки уведомлений. Проверено, что SMTP-клиент использует корректные параметры (хост, порт, таймаут 15 секунд), выполняет starttls и логин, а также отправляет сообщение (test_delivery_senders_and_filters_unit.py). Для WebPush протестирована отправка уведомлений без обязательного наличия публичного VAPID-ключа (допустим только приватный ключ с указанием subject).

Протестирован модуль уведомлений. Проверена функция фильтрации по диапазонам значений (_fits_range) для корректной обработки границ.

Протестирована обработка групп доставки: успешная отправка email-уведомлений и обработка ошибок WebPush с кодом 410 (Gone), при котором подписка автоматически деактивируется (`test_notification_pipeline_unit.py`).

Протестировано кэширование списка объявлений. Проверена логика построения ключа кэша в зависимости от параметров запроса (фильтры, сортировка, лимит). Кэшируются только запросы первой страницы с «горячими» фильтрами (цена, тип создателя); запросы со смещением или нестандартными фильтрами кэшированию не подлежат (`test_listing_cache_unit.py`). Также проверено поведение сервиса при попадании в кэш, промахе и ошибке чтения из Redis (`test_listings_service_cache_unit.py`).

Протестирована работа скраперов. Проверена загрузка всех трёх провайдеров («Авито», «Циан», «Домклик») через реестр, а также фильтрация по имени провайдера с учётом регистра и пробелов. Для каждого скрапера протестирован парсинг HTML с извлечением данных из встроенных JavaScript-объектов (`window._cianConfig`, `window.__SSR_STATE__`) и из DOM-элементов. Проверено корректное определение типа создателя объявления (собственник или агентство) (`test_scrapers.py`).

Протестирован API. С помощью `TestClient` проверены: требование аутентификации для защищённых эндпоинтов (возврат 401), создание настроек уведомлений с валидацией каналов (хотя бы один канал должен быть включён) и допустимых значений `creator_types`, запуск пайплайна уведомлений и задач парсинга только для пользователей с правами `staff` (возврат 403 для обычных пользователей), получение статуса фоновых задач: для известного `task_id` возвращается результат, для неизвестного – 404 (`test_notifications_api.py`, `test_scraping_api.py`).

Протестирована работа фоновых задач. Проверено, что задача `run_full_pipeline_task` агрегирует результаты вызовов функций сопоставления, материализации и отправки уведомлений, а также корректно логирует события начала и окончания (`test_notifications_task.py`). Аналогично

для задачи `run_all_scrapers_task` проверена сериализация результата (`ScrapeRunResult`) в словарь с подсчётом созданных, обновлённых и неудачных объявлений (`test_scraping_task.py`).

Протестирована работа JWT-токенов. Проверены функции хеширования паролей (`hash_password`, `verify_password`). Протестирована выдача предупреждения при использовании короткого секрета для HS256 (менее 32 байт). Проверено, что access-токен не может быть использован как refresh-токен, а также что refresh-токен позволяет получить новый access-токен даже после истечения старого (`test_security_and_signin.py`, `test_jwt_security_warning.py`).

В результате все перечисленные тесты успешно выполняются в среде непрерывной интеграции. Общее количество модульных тестов превышает 50. Тестирование выявило и помогло устранить несколько ошибок на ранних этапах разработки, связанных с некорректной обработкой границ диапазонов, неправильным формированием URL для объявлений и отсутствием обработки ошибок WebPush. Поддержание актуальности тестов при добавлении новой функциональности является обязательным требованием при разработке.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был разработан сервис для подбора недвижимости «Sweet Neighbors Club», автоматизирующий процесс мониторинга объявлений с крупнейших российских агрегаторов — «Авито», «Циан» и «Домклик». Результаты работы подтверждают достижение всех поставленных во введении целей и задач.

Проведён анализ предметной области, выявлены ключевые особенности рынка недвижимости: высокая стоимость объектов, сезонность спроса, неоднородность данных и быстрая оборачиваемость ликвидных предложений. Определены основные проблемы традиционного поиска — необходимость ручного просмотра нескольких сайтов, информационная перегрузка, высокий риск пропуска выгодных предложений и ограниченные возможности фильтрации. Выполнен обзор существующих решений, проанализированы функциональные возможности «Авито Недвижимость», «Циан» и «Домклик», их фильтры, системы уведомлений и ограничения. Сделан вывод о том, что каждый из этих сервисов замкнут на собственных данных, что заставляет пользователей использовать несколько площадок одновременно. Разрабатываемая система устраняет этот недостаток за счёт объединения данных из трёх источников, единых параметров фильтрации и централизованного управления уведомлениями.

Сформулированы функциональные и нефункциональные требования. В работе приведены 14 функциональных требований, охватывающих сбор данных, фильтрацию, уведомления, регистрацию и веб-интерфейс, а также 5 нефункциональных требований, включая периодичность обновления (не реже 10 минут), доступность (не менее 90%) и масштабируемость. Обоснован выбор технологического стека и архитектурных решений: для бэкенда выбран FastAPI (высокая производительность, асинхронность), база данных — PostgreSQL (поддержка JSONB, ACID), очереди задач — Celery, брокер сообщений — Redis. Фронтенд реализован на React, мобильная версия — как

Progressive Web App (PWA). Архитектура включает модули сбора, обработки, базы данных, API, брокера сообщений, кэша, уведомлений и веб-интерфейса.

Описан состав работ и этапы реализации проекта. Разработка велась по гибкой методологии Agile с двухнедельными спринтами. Составлено 5 спринтов (включая подготовительный), определены задачи каждого этапа. Методология позволила быстро адаптироваться к изменениям в структурах данных внешних источников и эффективно распределить ограниченные ресурсы команды (2 разработчика, 2 месяца). В практической части выполнено проектирование базы данных PostgreSQL (9 таблиц, ER-диаграмма, связи, защита от дублирования), разработка REST API на FastAPI (JWT-аутентификация, эндпоинты для регистрации, работы с объявлениями, уведомлениями и парсингом, асинхронные задачи Celery), разработка скраперов для трёх источников с парсингом JSON из встроенных JavaScript-объектов и DOM-элементов, настройка PWA для получения push-уведомлений на мобильных устройствах, реализация модуля уведомлений с отправкой по email и WebPush с обработкой ошибок (код 410 — деактивация подписки), а также написание комплекса автоматических модульных тестов (более 50 тестов), охватывающих репозитории, сервисы, API, скраперы, кэш, безопасность и фоновые задачи.

Все поставленные задачи выполнены в полном объёме. Разработанный сервис позволяет пользователям автоматически отслеживать новые объявления о недвижимости на трёх крупнейших площадках, настраивать гибкие параметры поиска и получать своевременные уведомления на электронную почту и мобильное устройство. Система готова к эксплуатации и может быть в дальнейшем расширена подключением новых источников, улучшением алгоритмов фильтрации и внедрением аналитики поведения пользователей. Таким образом, цель работы — проектирование и реализация информационной системы мониторинга объявлений недвижимости — достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Авито : [сайт]. – Москва, 2007. – URL: <https://www.avito.ru/voronezh/nedvizhimost> (дата обращения: 24.03.2026). – Текст : электронный.
2. Циан : [сайт]. – Москва, 2001. – URL: <https://www.cian.ru/> (дата обращения: 24.03.2026). – Текст : электронный.
3. Домклик : [сайт]. – Москва, 2018. – URL: <https://domclick.ru/> (дата обращения: 24.03.2026). – Текст : электронный.
4. ScrumTrek : [сайт]. – Москва, 2010. – URL: <https://scrumtrek.ru/blog/agile-scrum/4029/metodologiya-agile/> (дата обращения: 22.04.2026). – Текст : электронный.